

Model 模型
Module 模塊

An overview of UML

Unified Modeling Language

1

1

The objectives of OOAD

- create codes in good architecture, **easily maintainable**
- create reusable codes (via inheritance)
- modeling software before building it
 - (models can be validated, verified....)
- delay implementation stage and find any potential problems/design flaws before implementation
- change the way of software development
- OOAD documents are used as a blue print, serving as a guidelines for implementation. 減少摸石頭過河過程中錯誤所帶來的金錢與時間延遲
- OOAD documents serve as a communication tool between managers/programmers

2

2

What UML is aimed for

- Architects design buildings. Builders use the designs to create buildings.
- The more complicated the building, the more critical the communication between architect and builder.
- Blueprints are the standard graphical language that both architects and builders must learn as part of their trade.

3

3

OOAD standard language UML

- UML (Unified Modeling Language)
 - It is intended to become the standard visual models for software engineering (千呼萬喚始出來的一種軟體工程語言)
 - It is not only used in OOAD but can be used in other software applications where programs are not OO(可視為軟體的工程圖標準 (不見得只是對物件導向程式有用))
 - 由物件導向學界的三位巨頭成果結合而成
- 3 amigos
 - Grady Booch (Rational Software)
 - Jim Rumbaugh (GE)
 - Ivar Jacobson
- In practice, UML is gradually accepted as a standard visual models. Many researches show that it is useful in practice. (近年來的推廣，業界接受度很高，多項研究調查顯示UML在業界實用上有用)
- It is an informal language, but useful.
- Although it is called a LANGUAGE, actually it is a set of diagram (visual models) (雖然它稱為語言，事實上只是一組標準圖形表示方式)

4

4

UML's goal

- In the past decade, the UML has emerged as the software blueprint language for analysts, designers, and programmers alike. It is now part of the software trade.
- The UML gives everyone from business analyst to designer to programmer a common vocabulary to talk about software design.

5

5

UML — A design and analysis language for communication

- Have you ever read other people's source code?
- Have you ever need to understand an existing system?
- from source code -- a bad start
- from specification (規格書)
 - 見林不見樹

6

6

NOTES

- UML is just a tool, like OOPL, not a methodology
- You understand UML $X \Rightarrow$ you know OOAD (知道UML語言，不代表你會OOAD)
- Just like a mechanic can understand a blueprint but does not mean he can design one. (正如同懂OOPL不代表你會寫出真正的OO程式)
- Now, writing OOP mostly use UML as a standard tool (但是進行OOAD現在都會用UML語言來做為工具 (早期沒有標準化之前，各種圖示都有人使用))
- More UML diagrams are released for specific application domain (基本上UML含有OOAD必要的功能，但是它是一套適用任何類型軟體工程的語言，例如real time系統。(有些功能正不斷的擴充中))

7

7

When to use it

- In a forward engineering,
 - Specs $\rightarrow$ modeling $\rightarrow$ coding $\rightarrow$
 - 在需求規格制訂後，採用UML工具進行建構一個model然後分析與設計。然後可以利用分析所得到的UML分析資料，輸入UML工具，產生程式碼模版讓程式設計人員進行細節加工。分析過程中的UML文件可以視為工程藍圖，以及說明文件。
- In a backward engineering (亡羊補牢)
 - Code $\rightarrow$ UML diagrams : use UML for documentation purpose (在程式寫出來之後，利用軟體工具或人工，將程式的UML架構圖繪製出來，補寫文字，作為說明文件，只是一個亡羊補牢的措施)

8

8

How to use it

- 在各式各樣的軟體工程工具裡面繪製UML圖，並產生程式碼模版
 - Rational rose
 - Borland together
 - ArgoUML
 -
- Other pure drawing tools
 - Visio
 -

9

9

What is analysis and design?

- If you don't feel learning something solid in the class, don't blame me. **Design can be hard!**
- analysis and design is a software engineering process aimed for large or project based software development.
- In small programs (involving one or two persons) $\rightarrow$ understand $\rightarrow$ think $\rightarrow$ coding
- In large project, you are **in big trouble** if you think you are smart and you can remember and manage anything in your brain

10

10

The right time to get training of SE or OOAD

- many-year experienced programmers
- EMBA (enterprise Master of Being an Asshole)
- not ALL the MBA degree can attract business man.
- Typical MBA
 - non-experience graduates
 - Overwhelm the students with case studies
 - learn valuable experience from case studies
 - avoid mistakes from case studies
 - However, no way this can replace in-field experience
- MSE degree in CMU
- Advanced OOA?

11

11

Is everything must be OO?

- Not quite true
- OOA is used to structure a system, not to derive algorithms to solve problems.
- Implementing algorithms/systems which are still in research object-orientedly could be waste of time
- Who knows what will come next?
 - Component techniques?
 - Software architecture?

12

12

UML history

- terms need to know
 - OOPSLA (Object-oriented programming, systems, languages, and applications)
 - OMG (object management group)
 - Smalltalk

13

13

History of UML

- key methodologists protest standardization
 - Old joke: what is the difference between a methodologist and a terrorist?
 - Answer, you can negotiate with a terrorist
- OOPSLA 94
 - Booch and Rumbaugh merge their method and declare "the method war is over – we won"
 - others suggest anti-booch coalition
 - 1996 UML 1.0 announced
 - now UML 1.3

14

14

UML, a modeling language

- UML define a notation and a meta-model
- How strictly should you stick to the modeling language?
 - CASE tool: more strict to get acceptable codes
 - for communication purpose: you can have a little more leeway

15

15

Why you are using UML to do Analysis and Design

- Communication
 - allow you to communicate certain concepts more clearly than the alternatives
 - Natural language is too imprecise
 - code is precise but too detailed
 - You are suggested to use UML to highlight important details
 - avoiding lose sight of the forest for the trees on a large project
 - serving as a brief document of the system

16

16

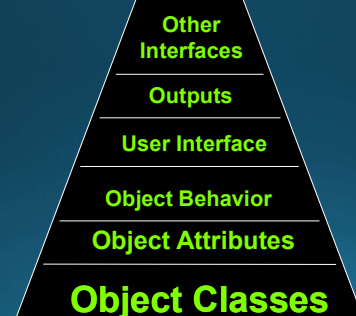
Why you are using UML to do Analysis and Design

- learning OO
 - Object languages allow advantages but don't provide them. by Tom Hadfield
- Communicating with domain experts
 - the biggest challenge of development
 - build the *right* system (just like in exam, you answer the right answer)

17

17

0 Jacobson's Three Types Interface Objects, Advantages:



18

18

① Jacobson's Three Types

Control Objects:

- Sometimes we find a method (subroutine, function) that uses data, and calls other methods, from **many different** objects and classes.
- These methods **don't easily fit or don't logically belong** in any of our **entity classes** or **interface classes**.

19

19

② 5.2. Classes and Classification

Additional Data-world Classes

Abstract Classes

- Often added to take advantage of inheritance and polymorphism
- Sometimes we invent an “artificial” superclass
- To make use of some common attributes or behavior

20

20

Discussion

- 分析的目的是為了要讓程式碼寫出來能夠充分的應付**未來的變化**。讓程式能夠順利的演化
- 好的分析不容易，大規模的分析通常必須由有經驗的人為之（可以讓叫沒經驗的程式員自行進行負責的小模組內的分析）
- 壞的分析等於浪費時間，反而另程式開發時程蹣跚
- 有分析通常比沒分析來的好很多，因為分析的過程中逼迫主事者了解絕大部分的細節，考慮所有可能發生的狀況，就算分析隨著程式設計而修改可以將風險降到最小，而不致延遲到 implementation

21

21

Discussion

- 好的分析！＝好的效能
通常具備好結構的軟體會具備有較差的效能，因為多了許多物件的新增，物件之間的呼叫。所以在 design 階段，有時候為了效能，又會合併 class
- 要程式員寫乾淨，好的程式碼不容易。如果能先行分析，可以逼迫程式設計人員按照架構寫出乾淨好維護的程式碼
- 好的分析可以造就好的分工, teamwork
- 成功的分析就是要能夠嗅出未來軟體可能演化的部分與方向。
- 針對任何案子，沒有所謂完美的分析。好壞只是程度上的差別，純粹看主事者要應付程式演化的問題到什麼程度
- 好的分析一定要落實到程式碼，沒有大量實做經驗背景的人所做的分析可能無法達到

22

22

Discussion

- 在你的軟體發展過程中，你認為花在分析與設計的時間要多少？
 - What type of software you build (what kind of market you are in)?
 - e.g. there is no need for analysis for most research
 - Is evolveability very important in your area?
 - is technology changing very fast in your area?
 - How long is your design/code typically out of date and thrown away?
 - what is the scale of your software?
 - what is the total cost of your software?
 - Are documentation/process important in your company?
 - Have your programmers high transition rate?
 - How much quality you care?
 - 6 month design/planning, 3 month coding, 3 month testing

23

23

An brief introduction of UML

- No single diagram are capable of describing software. (軟體工程幾十年的重要體認就是我們沒有辦法用單一種的圖像來表示軟體)
- There are several aspects of software. Must be dealt by several kind of diagrams. (軟體有許多面向，需要以不同的圖形表示方式來解決)
 - [Use case diagrams](#)
 - [Class diagrams](#)
 - [Object diagrams](#)
 - [Sequence diagrams](#)
 - [Collaboration diagrams](#)
 - [Statechart diagrams](#)
 - [Activity diagrams](#)
 - [Component diagrams](#)
 - [Deployment diagrams](#)

24

24

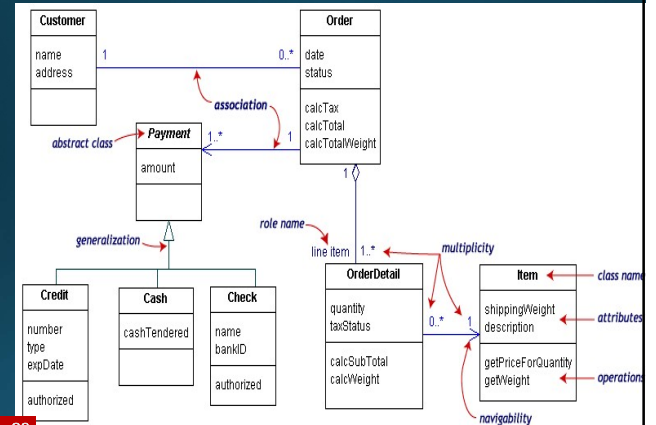
Use Case Diagram



25

25

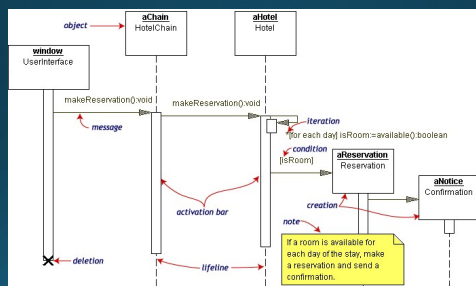
class diagram



26

26

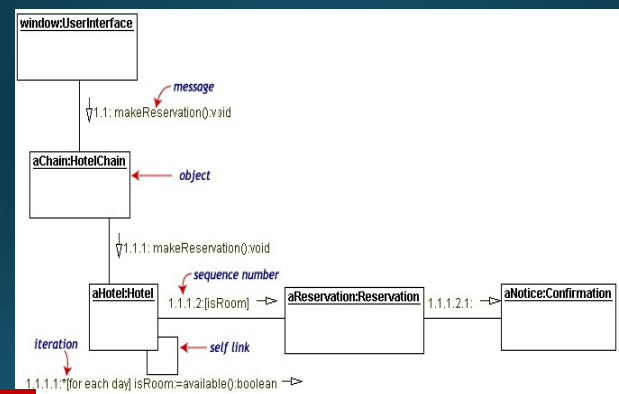
Sequence diagram



27

27

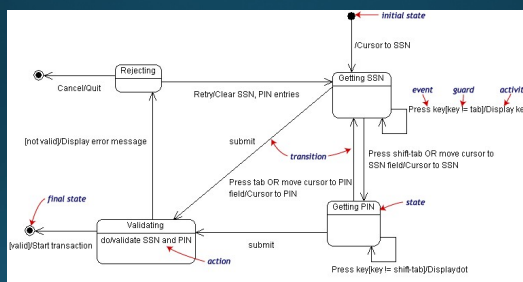
Collaboration diagram



28

28

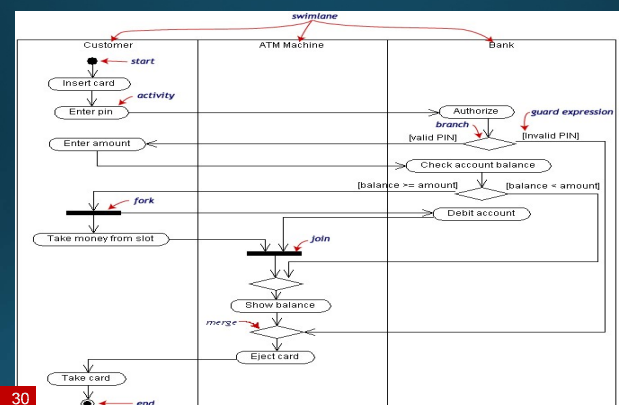
state chart



29

29

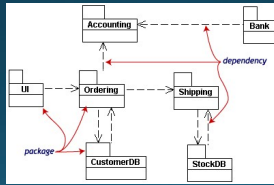
Activity diagram



30

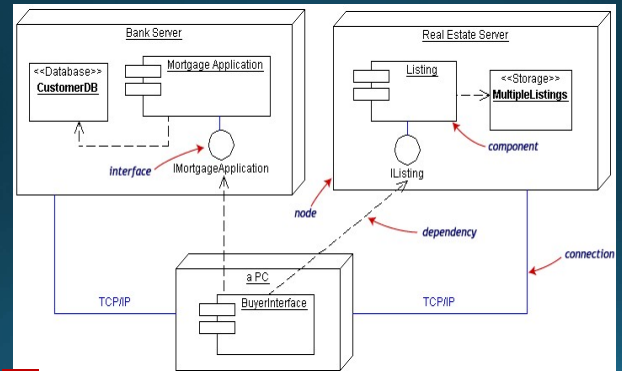
30

package and object diagram



31

Component and deployment diagram



32

31

32